



Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра суперкомпьютеров и квантовой информатики

Повжик Юрий Максимович

Сравнение средств декомпозиции графов

Научный руководитель:

доцент, к.ф.-м.н.,

Жуков Константин Андреевич

Москва, 2026

Аннотация

В работе рассматриваются современные методы параллельной декомпозиции графов в системах с распределённой памятью. Разработан инструмент проведения сравнительного анализа библиотек ParMETIS, PT-Scotch, ParHIP, dKaMinPar, Zoltan и Zoltan2. Реализованы средства параллельного чтения графов в формате CSR и вычисления метрик качества разбиения, включая число разрезанных рёбер, дисбаланс нагрузки и топологическую связность разделов. Разработанная система проводит оценку эффективности библиотек, выбор которой позволяет оптимизировать высокопроизводительные вычисления на графах.

Содержание

1	Введение.....	4
2	Постановка задачи	5
3	Рассматриваемые библиотеки разбиения графов.....	6
3.1	ParMETIS	7
3.2	PT-Scotch.....	8
3.3	KaHIP	10
3.4	KaMinPar	12
3.5	Zoltan (PHG)	14
3.6	Zoltan2 (Sphynx)	16
4	Обзор похожих работ и существующих решений.....	19
5	Метрики сравнения качества разбиения.....	21
5.1	Метрики балансировки нагрузки	21
5.2	Коммуникационные метрики.....	21
5.3	Топологические метрики	22
5.4	Временные характеристики.....	22
6	Программная реализация	24
6.1	Модуль чтения и подготовки графа.....	25
6.2	Модуль реализации алгоритмов разбиения	27
6.3	Модуль сбора и анализа метрик качества.....	31
7	Результаты тестирования	35
7.1	Сравнение алгоритмов по коммуникационным и топологическим метрикам ..	36
7.2	Сравнение алгоритмов по балансировке нагрузки.....	40
7.3	Сравнение временных характеристик.....	43
7.4	Итоги сравнения	45
8	Заключение.....	47
9	Список литературы	48

1 Введение

В условиях развития высокопроизводительных вычислений (HPC) графы широко применяются в качестве универсальной модели представления сложных структур данных и взаимосвязей между объектами. Графовые модели используются при моделировании физических процессов, анализе социальных сетей, обработке больших данных, решении задач биоинформатики и в ряде других прикладных областей. Увеличение объёмов обрабатываемых данных и сложности вычислений приводит к необходимости использования многопроцессорных систем с распределённой памятью.

Одним из ключевых факторов эффективности параллельной обработки графов является качество их декомпозиции. Разделение графа должно обеспечивать равномерное распределение вычислительной нагрузки между вычислительными узлами и одновременно минимизировать объём межпроцессорных обменов, возникающих из-за разрезанных рёбер. Задача оптимального разбиения графа относится к классу NP-полных [1], поэтому на практике применяются эвристические алгоритмы, реализованные в специализированных программных библиотеках.

Существует большое количество библиотек для параллельной декомпозиции графов, включая ParMETIS [2], PT-Scotch [3], KaHIP [4], KaMinPar [5], Zoltan [6] и Zoltan2 [7], использующих различные алгоритмические подходы и методы оптимизации. Однако эффективность этих решений существенно зависит от структуры графа, числа разделов и критериев качества разбиения. Отсутствие единой среды для их тестирования и сопоставления в одинаковых условиях затрудняет проведение объективного сравнительного анализа и выбор наиболее подходящего инструмента для конкретных задач высокопроизводительных вычислений.

Целью данной работы является исследование и сравнительный анализ библиотек параллельной декомпозиции графов, а также разработка программной системы для автоматизированного тестирования и оценки качества разбиения графов в среде MPI [8].

2 Постановка задачи

Целью данной работы является разработка программной системы для сравнительного анализа современных библиотек параллельной декомпозиции графов и исследования эффективности реализованных в них алгоритмов.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Разработать модуль параллельного ввода и распределённого хранения графов, обеспечивающий считывание графов большого объёма в формате CSR [9] и их корректное распределение между MPI-процессами.
2. Реализовать средства предварительной обработки графов.
3. Спроектировать унифицированный программный интерфейс для интеграции библиотек ParMETIS, PT-Scotch, ParHIP, dKaMinPar, Zoltan и Zoltan2 с возможностью настройки параметров декомпозиции.
4. Разработать систему вычисления и сбора метрик качества разбиения графов, включающую оценку числа разрезанных рёбер, дисбаланса нагрузки, количества граничных вершин, связности разделов и характеристик коммуникационной топологии.
5. Провести сравнительное исследование библиотек декомпозиции на графах различной структуры и размерности, выполнив анализ времени работы алгоритмов, их масштабируемости и качества получаемых разбиений.

Результатом работы должна стать программная система на языке C++ [10], предназначенная для автоматизированного тестирования библиотек параллельной декомпозиции графов в среде MPI и формирования детализированных отчётов, пригодных для последующего анализа и визуализации результатов.

3 Рассматриваемые библиотеки разбиения графов

Для проведения сравнительного анализа были выбраны наиболее актуальные и широко используемые в академической и инженерной среде программные комплексы, поддерживающие параллельные вычисления на основе MPI. Такой выбор позволяет не только оценить качество разбиения графов, но и исследовать масштабируемость алгоритмов в распределённой вычислительной среде.

Сами библиотеки можно классифицировать по используемому алгоритмическому подходу и модели представления графа. Данное разделение позволяет более корректно сопоставлять методы разбиения не только по качеству получаемого разбиения, но и по их вычислительной природе и характеру параллельной реализации.

В рамках исследования выделяются следующие основные классы методов:

- Многоуровневые алгоритмы разбиения графов [11], к которым относятся ParMETIS, PT-Scotch, KaHIP и KaMinPar. Данные методы основаны на последовательном огрублении графа, построении начального разбиения и последующем его уточнении.
- Гиперграфовые методы, представленные библиотекой Zoltan (алгоритм RNG [12]), которые позволяют более точно моделировать коммуникационные структуры в распределённых вычислениях за счёт использования гиперграфовой модели.
- Спектральные методы разбиения графов, реализованные в модуле Zoltan2 (Sphynx [13]), основанные на анализе спектральных свойств лапласиана графа и переходе к разбиению в спектральном пространстве.

Такое разделение позволяет провести сравнительный анализ методов, относящихся к различным математическим моделям графов и различным стратегиям оптимизации, включая балансировку нагрузки, минимизацию разреза и масштабируемость в распределённой среде MPI.

3.1 ParMETIS

ParMETIS является параллельным развитием библиотеки METIS [14], разработанной в Университете Миннесоты группой Джорджа Кариписа. Библиотека ориентирована на задачи разбиения больших неориентированных графов и широко применяется в системах высокопроизводительных вычислений (HPC), особенно при распределении сеток и матриц между вычислительными узлами.

В основе ParMETIS лежит многоуровневый подход (multilevel graph partitioning), включающий три основных этапа:

1. последовательное огрубление графа (coarsening);
2. разбиение уменьшенного графа;
3. постепенное уточнение решения при восстановлении исходной структуры (uncoarsening/refinement).

Для параллельного разбиения используется MPI, что позволяет эффективно масштабировать вычисления на кластерах. В работе применяется функция `ParMETIS_V3_PartKway`, реализующая прямое k-way разбиение с минимизацией количества разрезанных рёбер (edge cut).

К основным преимуществам библиотеки относятся:

1. высокая скорость работы;
2. хорошая масштабируемость;
3. стабильное качество разбиения;
4. широкое распространение в научных пакетах.

Недостатком ParMETIS является сравнительно ограниченная гибкость стратегий балансировки и зависимость качества результата от структуры входного графа.

Библиотека выбрана в качестве базового эталона производительности, поскольку фактически является промышленным стандартом для MPI-приложений.

3.2 PT-Scotch

PT-Scotch является параллельной версией библиотеки Scotch [2], разрабатываемой исследовательской группой LaBRI (Laboratoire Bordelais de Recherche en Informatique, Франция). Библиотека предназначена для разбиения графов, распределения вычислительных задач и отображения графовых структур на архитектуру вычислительных систем. PT-Scotch активно используется в задачах высокопроизводительных вычислений, включая распределение сеток в методах конечных элементов [15] и балансировку нагрузки в MPI-приложениях.

В отличие от ParMETIS, PT-Scotch в значительной степени опирается на подход рекурсивного дуального деления пополам (recursive bipartitioning) [16]. Алгоритм последовательно разбивает граф на две части до тех пор, пока не будет достигнуто требуемое число подграфов. Такой подход обеспечивает хорошую управляемость процесса балансировки и позволяет гибко адаптировать стратегию разбиения под конкретный тип задачи.

Архитектура библиотеки основана на многоуровневой схеме (multilevel partitioning), включающей:

1. огрубление графа;
2. построение начального разбиения;
3. локальное уточнение решения.

Для обмена данными между процессами используется MPI, благодаря чему библиотека способна эффективно работать на распределённых вычислительных системах и масштабироваться на большое количество процессов.

В рамках исследования используются различные стратегии разбиения:

- SCOTCH_STRATQUALITY — приоритет качества разбиения;
- SCOTCH_STRATSPEED — минимизация времени работы;
- SCOTCH_STRATBALANCE — усиленный контроль балансировки нагрузки.

Использование нескольких стратегий позволяет исследовать компромисс между качеством разреза графа и вычислительными затратами алгоритма.

К преимуществам PT-Scotch относятся:

- высокая гибкость настройки алгоритмов;
- эффективная балансировка нагрузки;
- поддержка отображения графов на топологию вычислительной системы;
- хорошая масштабируемость в MPI-среде.

Среди ограничений библиотеки можно отметить более сложную настройку параметров по сравнению с ParMETIS, а также зависимость качества результата от выбранной стратегии разбиения.

Библиотека включена в исследование как представитель подхода рекурсивного разбиения и как один из наиболее распространённых инструментов для задач балансировки нагрузки в распределённых НРС-приложениях.

3.3 KaHIP

KaHIP (Karlsruhe High Quality Partitioning) представляет собой комплекс алгоритмов для разбиения графов, разработанный в Институте теоретической информатики Технологического института Карлсруэ (KIT, Германия). Основной целью библиотеки является получение разбиений максимально высокого качества при минимальном количестве разрезанных рёбер и строгом соблюдении ограничений балансировки.

В отличие от классических библиотек, ориентированных преимущественно на скорость работы, KaHIP делает акцент на качестве итогового разбиения. Для этого библиотека объединяет несколько алгоритмических подходов, включая:

- многоуровневое разбиение (multilevel partitioning);
- методы локального уточнения;
- потоковые алгоритмы;
- эволюционные и меметические стратегии оптимизации [17].

Алгоритм работы библиотеки включает стандартные этапы огрубления графа, начального разбиения и последующего уточнения, однако особое внимание уделяется именно фазе refinement. Для локального улучшения разбиения используются модифицированные варианты алгоритма Фидучча–Маттейсеса (FM-refinement) [18], а также методы потоковой оптимизации, позволяющие уменьшать размер разреза без нарушения баланса частей.

Для параллельной обработки распределённых графов в работе используется компонент ParHIP [3], обеспечивающий поддержку MPI и выполнение разбиения на вычислительных кластерах. ParHIP ориентирован на обработку сверхбольших графов и использует распределённую многоуровневую схему, минимизирующую объём межпроцессорных коммуникаций.

Одной из особенностей KaHIP является наличие большого числа готовых конфигураций, ориентированных на различные сценарии использования:

- fast — ускоренное построение разбиения;

- есо — компромисс между скоростью и качеством;
- strong — получение разбиения высокого качества;
- fastsocial и ultrafastsocial — специализированные режимы для социальных графов.

К основным преимуществам KaHIP относятся:

- высокое качество разбиения;
- эффективные методы локального уточнения;
- поддержка распределённых вычислений через MPI;
- хорошая масштабируемость на графах большого размера;
- наличие специализированных стратегий для различных типов графов.

Недостатком библиотеки является сравнительно более высокое время работы по сравнению с ParMETIS и PT-Scotch, особенно при использовании режимов, ориентированных на максимальное качество.

KaHIP включена в сравнительный анализ как библиотека, ориентированная прежде всего на качество разбиения, и используется в качестве одного из эталонов минимизации размера разреза графа.

3.4 KaMinPar

KaMinPar (Karlsruhe Minimal Partitioning) — современная библиотека для разбиения графов, разрабатываемая исследовательской группой Технологического института Карлсруэ (KIT). Библиотека ориентирована на обработку сверхбольших графов и эффективное масштабирование на современные многоядерные и кластерные архитектуры. Основной целью KaMinPar является достижение высокой производительности при сохранении качества разбиения, сопоставимого с лучшими существующими решениями.

В основе библиотеки лежит глубокая многоуровневая схема (deep multilevel partitioning) [19], отличающаяся от классических подходов значительно большим числом уровней огрубления графа. Такой подход позволяет уменьшить размер локальных задач и повысить эффективность алгоритмов уточнения на каждом этапе восстановления графа.

Алгоритм работы KaMinPar включает:

1. последовательное огрубление графа;
2. построение начального разбиения на наиболее грубом уровне;
3. многоэтапное уточнение при восстановлении структуры графа.

Особое внимание в библиотеке уделяется параллельной обработке данных. KaMinPar использует гибридный подход к распараллеливанию, сочетающий:

- распределённые вычисления на основе MPI;
- многопоточную обработку внутри вычислительных узлов.

Это позволяет эффективно использовать как межузловой, так и внутроузловой параллелизм современных HPC-систем.

Для уменьшения накладных расходов библиотека применяет:

- NUMA-aware оптимизации памяти [20];
- локализацию данных;

- минимизацию межпроцессорных коммуникаций;
- специализированные структуры хранения графов.

Одной из ключевых особенностей KaMinPar является ориентация на разбиение графов на очень большое число частей (many-way partitioning), что особенно важно для задач распределённых вычислений на тысячах MPI-процессов.

В рамках исследования библиотека тестируется с использованием различных конфигураций:

- default — стандартный режим работы;
- strong — режим, ориентированный на получение более качественного разбиения.

Сравнение различных режимов позволяет оценить влияние параметров алгоритма на качество разреза и время выполнения вычислений.

К преимуществам KaMinPar относятся:

- высокая масштабируемость;
- эффективная работа на графах большого размера;
- оптимизация под современные многоядерные архитектуры;
- низкие накладные расходы при параллельной обработке;
- хорошее соотношение между скоростью и качеством разбиения.

Среди ограничений библиотеки можно отметить сравнительно небольшую зрелость проекта по сравнению с ParMETIS или PT-Scotch, а также меньшее количество исследований и практических интеграций в прикладные HPC-пакеты.

KaMinPar включена в исследование как современный представитель высокомасштабируемых многоуровневых алгоритмов разбиения, ориентированных на работу в распределённых MPI-средах и обработку сверхбольших графов.

3.5 Zoltan (PHG)

Zoltan — это библиотека динамической балансировки нагрузки и разбиения данных, разработанная в национальных лабораториях Sandia National Laboratories (США). Библиотека ориентирована на задачи высокопроизводительных вычислений и широко применяется в научных пакетах для моделирования физических процессов, решения систем линейных уравнений и распределённых вычислений на суперкомпьютерах.

Одним из ключевых компонентов библиотеки является алгоритм **PHG** (Parallel HyperGraph Partitioning), реализующий параллельное разбиение гиперграфов. В отличие от классических графовых моделей, где ребро соединяет только две вершины, гиперграф позволяет одному гиперребру связывать сразу множество вершин. Такая модель более точно отражает структуру коммуникаций в распределённых вычислениях, особенно при работе с разреженными матрицами и сеточными методами.

Алгоритм PHG ориентирован на минимизацию объёма межпроцессорного обмена данными. Это достигается за счёт:

- построения гиперграфовой модели вычислений;
- балансировки вычислительной нагрузки между процессами;
- минимизации количества «разделённых» гиперрёбер.

PHG использует многоуровневый подход к разбиению, включающий:

1. огрубление гиперграфа;
2. построение начального разбиения;
3. локальное уточнение решения.

Для параллельного выполнения библиотека использует MPI, что позволяет эффективно выполнять разбиение распределённых структур данных на вычислительных кластерах.

Несмотря на использование гиперграфовой модели, PHG может применяться и для обычных графов. В этом случае стандартный граф

преобразуется в гиперграфовое представление, что позволяет использовать преимущества гиперграфового подхода даже для классических задач graph partitioning. Это особенно полезно при моделировании коммуникационных затрат в задачах распределённых матриц и сеточных вычислений, где обычная графовая модель может недостаточно точно отражать структуру обменов между процессами.

Главным преимуществом гиперграфового подхода является более точная оценка реальных коммуникационных затрат по сравнению с обычными графовыми моделями. Это особенно важно для задач:

- распределения разреженных матриц;
- параллельных методов конечных элементов;
- научных симуляций с неравномерной коммуникацией между процессами.

К преимуществам RNG относятся:

- высокая точность моделирования коммуникаций;
- эффективная балансировка нагрузки;
- хорошая масштабируемость в MPI-среде;
- широкое применение в научных НРС-приложениях.

Недостатком метода является более высокая вычислительная сложность по сравнению с классическими графовыми алгоритмами, а также повышенные требования к памяти при построении гиперграфовой модели.

Алгоритм RNG включён в исследование как представитель гиперграфового подхода к разбиению, позволяющего оценить эффективность более точного моделирования коммуникаций в распределённых вычислениях.

3.6 Zoltan2 (Sphynx)

Zoltan2 является развитием библиотеки Zoltan и входит в состав фреймворка Trilinos, предназначенного для высокопроизводительных научных вычислений. В отличие от оригинальной версии, Zoltan2 использует современную шаблонную архитектуру C++ и обеспечивает более тесную интеграцию с распределёнными структурами данных и HPC-библиотеками, что делает её более гибкой и удобной для использования в современных MPI-приложениях.

В рамках данного исследования рассматривается модуль **Sphynx**, реализующий спектральный метод разбиения графов (spectral graph partitioning). Данный алгоритм ориентирован на гибридные вычислительные системы, поддерживающие распределённую память и GPU-ускорение, и является одним из первых спектральных partitioning-решений, работающих в MPI-среде с поддержкой ускорителей.

В отличие от классических рекурсивных спектральных методов, Sphynx вычисляет сразу $\log_2(K)$ собственных векторов, где K — требуемое число частей разбиения. После этого каждая вершина графа отображается в спектральное пространство, а итоговое разбиение выполняется с помощью алгоритма Multi-Jagged (MJ) [21], реализованного в Zoltan2. Такой подход позволяет перейти от рекурсивной схемы к глобальному спектральному представлению графа.

Для вычисления собственных векторов используется алгоритм LOBPCG (Locally Optimal Block Preconditioned Conjugate Gradient) [22], реализованный в пакете Anasazi [23]. Его важным преимуществом является возможность применения различных предобуславливателей, которые существенно влияют на скорость сходимости и устойчивость вычислений. В Sphynx поддерживаются следующие типы предобуславливателей:

- `jacobi` — диагональный предобуславливатель;
- `polynomial` — полиномиальный предобуславливатель;
- `muelu` — алгебраический многосеточный метод (AMG) [24].

Использование `tuelu` особенно эффективно для структурированных графов и позволяет существенно ускорить процесс сходимости спектрального решателя.

Отдельное внимание в алгоритме уделяется выбору спектральной постановки задачи, которая задаётся параметром `sphynx_problem_type`. От неё зависит способ формирования лапласиана графа и, соответственно, метрика спектрального пространства:

- `combinatorial` — классический комбинаторный лапласиан;
- `generalized` — обобщённая спектральная задача;
- `normalized` — нормализованный лапласиан [25].

Эти варианты соответствуют различным нормам и способам взвешивания структуры графа, что позволяет адаптировать алгоритм к неоднородным и плотным графам, а также уменьшать влияние вершин с высокой степенью.

Использование различных конфигураций позволяет исследовать влияние как спектральной модели, так и численных параметров решателя на качество разбиения и скорость работы алгоритма.

Sphynx поддерживает:

- распределённые вычисления через MPI;
- гибридную `shared/distributed-memory` архитектуру;
- GPU-ускорение;
- многокритериальную балансировку вершин.

Основной целью алгоритма является минимизация количества разрезанных рёбер (`edge cut`) при сохранении баланса нагрузки между частями графа.

К основным преимуществам Sphynx относятся высокая точность разбиения за счёт учёта глобальной структуры графа, гибкость спектральной

настройки, поддержка GPU и тесная интеграция с экосистемой Trilinos. В то же время к недостаткам относятся высокая вычислительная сложность спектрального этапа, повышенные требования к памяти, а также необходимость структурной симметричности и связности входного графа.

В рамках исследования Sphynx рассматривается как представитель спектрального класса алгоритмов, позволяющий сравнить классические многоуровневые методы с подходами, основанными на глобальном спектральном анализе структуры графа.

4 Обзор похожих работ и существующих решений

Задача сравнения эффективности библиотек разбиения графов на протяжении последних десятилетий является одной из ключевых в области высокопроизводительных вычислений (HPC). Значительное количество исследований посвящено разработке и сравнительному анализу алгоритмов, направленных на минимизацию межпроцессорных коммуникаций при сохранении баланса вычислительной нагрузки.

Одним из наиболее ранних и системообразующих исследований в данной области стал 10-й конкурс DIMACS Implementation Challenge [26], проведённый в 2012 году. В рамках данного конкурса были формализованы подходы к оценке алгоритмов разбиения графов и проведено масштабное сравнение существующих решений. По его результатам библиотеки ParMETIS и PT-Scotch были признаны одними из наиболее эффективных и стали де-факто стандартами для промышленных и научных задач. Основное внимание в исследованиях того периода уделялось классическим метрикам качества, таким как число разрезанных рёбер (edge cut) и время выполнения алгоритма, на стандартных наборах тестовых графов, включая семейство Walshaw Benchmark [27].

В дальнейшем развитие области было связано с появлением более сложных эвристических методов и расширением требований к качеству разбиения. Существенный вклад в развитие алгоритмов внесли библиотеки KaHIP и семейство инструментов Zoltan/Zoltan2, предложившие более сложные стратегии локального улучшения, эволюционные методы оптимизации и альтернативные модели представления графов, включая спектральные и гиперграфовые подходы. Эти решения позволили существенно повысить качество разбиения, однако привели к усложнению алгоритмической структуры и увеличению вычислительных затрат.

Несмотря на наличие значительного числа работ в данной области, современные исследования обладают рядом ограничений, что обуславливает актуальность настоящей работы.

Во-первых, большинство сравнительных исследований было проведено

до появления новых высокоэффективных библиотек, таких как KaMinPar, ориентированных на обработку сверхбольших графов в современных многопроцессорных и распределённых системах. В результате отсутствует достаточное количество независимых сравнений, в которых новые решения сопоставляются с классическими библиотеками в единых условиях тестирования.

Во-вторых, в существующих работах, как правило, используется ограниченный набор метрик, преимущественно включающий edge cut и время выполнения алгоритма. Однако для практических НРС-приложений этого недостаточно, поскольку не учитываются структурные характеристики полученных разбиений. В данной работе дополнительно используются топологические метрики, включая число связных компонент внутри каждого раздела и характеристики межраздельного соседства, что позволяет более точно оценивать влияние разбиения на коммуникационные издержки.

В-третьих, многие исследования ориентированы на отдельные семейства алгоритмов и не обеспечивают их прямого сравнения в рамках единой экспериментальной среды. В данной работе реализован унифицированный подход к тестированию, при котором алгоритмы различных классов — многоуровневые, спектральные и гиперграфовые — оцениваются на одном и том же наборе входных графов с использованием единой программной инфраструктуры на базе MPI и Tpetra [28].

Наконец, значительная часть существующих результатов основана на аппаратной базе, не отражающей характеристики современных вычислительных кластеров, что снижает актуальность таких сравнений в контексте современных НРС-систем.

Таким образом, актуальность данной работы обусловлена необходимостью обновления и расширения существующих сравнительных исследований с учётом современных алгоритмов разбиения графов, новых программных решений, а также расширенного набора метрик качества, позволяющих более полно оценивать поведение алгоритмов в условиях современных распределённых вычислительных систем.

5 Метрики сравнения качества разбиения

Для проведения сравнительного анализа библиотек разбиения графов в работе используется набор метрик, позволяющих оценить качество получаемых декомпозиций с точки зрения балансировки нагрузки, объёма межпроцессорных коммуникаций, структурных свойств разделов и временной эффективности алгоритмов. Совместное использование данных характеристик обеспечивает комплексную оценку алгоритмов в условиях параллельных вычислений на основе MPI.

5.1 Метрики балансировки нагрузки

Балансировка нагрузки является ключевым требованием к алгоритмам разбиения графов, поскольку неравномерное распределение данных между процессами приводит к простоям вычислительных узлов и снижению общей производительности.

Дисбаланс по вершинам (Vertex Imbalance) характеризует отклонение числа вершин в каждом разделе от идеального среднего значения. Чем ниже значение данной метрики, тем более равномерно распределена вычислительная нагрузка между процессами.

Дисбаланс по рёбрам (Edge Imbalance) отражает неравномерность распределения рёбер графа между разделами. Данная метрика особенно важна для задач, в которых вычислительная сложность определяется количеством связей, например при обработке разреженных матриц или моделировании графовых процессов.

5.2 Коммуникационные метрики

Коммуникационные метрики позволяют оценить объём межпроцессорных взаимодействий, возникающих при выполнении параллельных вычислений на разбиённом графе.

Разрез рёбер (Edge Cut) определяется как количество рёбер, соединяющих вершины, принадлежащие различным разделам. Данная метрика является одной из основных характеристик качества разбиения: уменьшение

edge cut напрямую снижает объём обмена данными между MPI-процессами.

Количество граничных вершин (Boundary Vertices) отражает число вершин, имеющих связи с вершинами других разделов. Увеличение данного показателя приводит к росту объёма синхронизационных данных, необходимых для корректного выполнения распределённых вычислений.

Количество соседних разделов (Neighbors Count) характеризует число различных разделов, с которыми взаимодействует каждый отдельный раздел графа. Чем выше значение данной метрики, тем сложнее коммуникационная структура и выше накладные расходы на передачу данных между процессами.

5.3 Топологические метрики

Топологические характеристики позволяют оценить внутреннюю структуру полученных разбиений и степень их связности, что важно для обеспечения локальности данных и эффективности кэш-использования.

Количество связных компонент в разделе (Connected Components per Part) показывает, насколько каждый раздел является связным подграфом. В идеальном случае каждый раздел должен представлять собой одну связную компоненту. Увеличение числа компонент свидетельствует о фрагментации разбиения, что может негативно влиять на эффективность вычислений.

Для вычисления данной характеристики используется алгоритм обхода графа с учётом распределённого характера данных и последующим объединением локально найденных компонент между MPI-процессами.

5.4 Временные характеристики

Временные метрики используются для оценки вычислительной эффективности рассматриваемых библиотек.

Время декомпозиции (Solve Time) отражает время выполнения непосредственно алгоритма разбиения графа внутри библиотеки. Данная метрика позволяет оценить вычислительную сложность алгоритма.

Общее время выполнения (Total Time) включает полный цикл обработки графа: загрузку данных, построение внутренних структур, выполнение разбиения и вычисление результирующих метрик. Данный показатель позволяет оценить суммарные накладные расходы при использовании конкретного программного комплекса.

6 Программная реализация

Программный комплекс разработан на языке C++ с использованием стандарта MPI (Message Passing Interface) для обеспечения выполнения вычислений в распределённой памяти и поддержки запуска на многопроцессорных вычислительных системах. Основной целью реализации являлось создание унифицированной программной среды, позволяющей выполнять сравнение различных библиотек разбиения графов в идентичных условиях эксперимента и обеспечивать автоматизированный сбор расширенного набора метрик качества.

При разработке особое внимание уделялось независимости экспериментальной инфраструктуры от конкретной библиотеки разбиения. Для этого была реализована единая система загрузки графов, подготовки распределённых структур данных, запуска алгоритмов декомпозиции и последующего анализа результатов. Такой подход позволяет минимизировать влияние особенностей пользовательской реализации на итоговые результаты сравнения.

В качестве базовой инфраструктуры для работы с распределёнными графовыми структурами используется библиотека Tpetra, входящая в состав фреймворка Trilinos. Tpetra обеспечивает средства для хранения распределённых графов и матриц, управления отображением данных между MPI-процессами, а также построения унифицированных интерфейсов взаимодействия с библиотеками разбиения.

Использование Tpetra позволило реализовать единый формат представления входных данных для всех рассматриваемых алгоритмов, включая многоуровневые, гиперграфовые и спектральные методы. Кроме того, интеграция с Trilinos обеспечила возможность прямого использования модулей Zoltan2 и Sphynx без дополнительного преобразования распределённых структур данных.

Программный комплекс состоит из трёх основных подсистем:

- подсистемы чтения и распределения входных графов;
- подсистемы запуска алгоритмов разбиения;

- подсистемы вычисления и анализа метрик качества.

Дальнейшие подразделы подробно описывают архитектуру и особенности реализации каждой из указанных компонентов.

6.1 Модуль чтения и подготовки графа

Корректность и производительность сравнительного анализа библиотек разбиения графов в значительной степени зависят от эффективности загрузки входных данных и их распределения между MPI-процессами. В связи с этим в рамках работы был реализован специализированный модуль чтения графов, основанный на собственном классе CSR, обеспечивающем параллельную загрузку данных, их распределение между процессами и подготовку структур к передаче во внешние библиотеки разбиения.

В качестве основного формата хранения входных данных используется формат CSR (Compressed Sparse Row), являющийся стандартным представлением разреженных графов и матриц в задачах высокопроизводительных вычислений. Основным преимуществом данного формата является компактное хранение структуры графа и эффективный доступ к спискам смежности вершин.

Структура CSR-представления включает:

- 1 заголовок файла, содержащий число вершин и рёбер графа;
- 2 массив указателей строк rows, определяющий границы списков смежности;
- 3 массив индексов cols, содержащий идентификаторы смежных вершин.

Такое представление обеспечивает линейное хранение графа в памяти и минимизирует накладные расходы при работе с крупными разреженными структурами данных.

Для обеспечения масштабируемости на больших графах модуль использует механизм параллельного ввода-вывода MPI-ИО. В отличие от последовательного подхода, при котором один процесс полностью считывает

файл и затем распределяет данные между остальными процессами, MPI-IO позволяет каждому процессу напрямую считывать только принадлежащую ему часть графа. Это существенно снижает требования к памяти и уменьшает накладные расходы на коммуникации.

Основная логика чтения реализована в методе `readCSR` и включает несколько этапов.

На первом этапе процесс с рангом 0 считывает метаданные файла — число вершин и рёбер — после чего данные распространяются между всеми MPI-процессами посредством операции `MPI_Bcast`.

Далее выполняется статическое распределение строк графа между процессами. Для этого используется схема равномерного деления с остатком, обеспечивающая максимально равномерное распределение нагрузки даже в случае, когда число строк не делится на количество процессов без остатка.

После вычисления локальных диапазонов каждый процесс определяет своё смещение относительно начала файла и выполняет коллективную операцию `MPI_File_read_at_all`. Использование коллективного чтения позволяет MPI-библиотеке оптимизировать доступ к файловой системе и уменьшить число независимых операций ввода-вывода.

После загрузки локальных фрагментов графа выполняется процедура нормализации массива `rows`. Поскольку в исходном файле указатели строк содержат глобальные смещения относительно всего массива рёбер, локальные структуры данных должны быть преобразованы к локальной системе координат.

Для этого из всех элементов локального массива `rows` вычитается значение первого элемента. В результате локальные индексы рёбер внутри процесса начинаются с нуля, что обеспечивает корректную передачу указателей во внешние библиотеки разбиения графов.

Различные библиотеки предъявляют разные требования к структуре входного графа. Например, классические многоуровневые библиотеки, такие как `ParMETIS` и `KaHIP`, предполагают отсутствие петель (диагональных элементов), тогда как спектральные методы, реализованные в `Zoltan2` (`Sphynx`), требуют наличия диагональных элементов для корректного построения матрицы

Лапласа.

Для поддержки различных сценариев в модуле реализованы специальные процедуры модификации структуры графа:

- `deleteDiagElems()` — удаление диагональных элементов;
- `addDiagElems()` — добавление отсутствующих диагональных элементов с сохранением сортировки списков смежности.

При добавлении диагоналей создаётся отдельная копия CSR-структуры (`rows_diag`, `cols_diag`), что позволяет динамически переключаться между графом с диагональными элементами и графом без них в зависимости от используемого алгоритма разбиения.

Для работы большинства распределённых библиотек разбиения требуется информация о глобальном распределении вершин между процессами. С этой целью реализован метод `computeVtxDist`, формирующий массив `vtxdist`.

Построение массива выполняется посредством операции `MPI_Allgather`, собирающей информацию о количестве локальных вершин на каждом процессе. После этого вычисляются префиксные суммы, определяющие диапазоны глобальных индексов, принадлежащих каждому MPI-процессу.

Полученный массив `vtxdist` используется библиотеками разбиения для определения владельца вершины по её глобальному индексу и организации межпроцессорных коммуникаций во время выполнения алгоритмов декомпозиции.

6.2 Модуль реализации алгоритмов разбиения

Центральным компонентом программного комплекса является модуль реализации алгоритмов разбиения, обеспечивающий взаимодействие с внешними библиотеками декомпозиции графов. Основной задачей данного модуля являлось создание унифицированного интерфейса над различными API библиотек разбиения, что позволило проводить сравнительный анализ алгоритмов в идентичных условиях и минимизировать влияние особенностей

пользовательской реализации на итоговые результаты.

Модуль реализован в классе `Partition`, который инкапсулирует логику подготовки данных, запуска алгоритмов и измерения временных характеристик.

Для автоматизации процесса тестирования в классе реализован универсальный метод `run`, принимающий в качестве аргумента тип алгоритма разбиения (`Type`). Такой подход позволяет в рамках единого сценария выполнения последовательно запускать различные библиотеки на одном и том же наборе входных данных.

Класс управляет общими параметрами декомпозиции:

- `nparts` — требуемое количество разделов;
- `imbalance` — допустимый коэффициент дисбаланса;
- `seed` — фиксированное начальное значение генератора случайных чисел.

Использование фиксированного значения `seed` обеспечивает воспроизводимость результатов и позволяет исключить влияние случайной инициализации на итоговое качество разбиения.

Для библиотек, использующих низкоуровневое представление графа в формате CSR, реализована передача прямых указателей на распределённые массивы:

- `vtxdist` — распределение вершин между MPI-процессами;
- `rows` — массив указателей строк;
- `cols` — массив списков смежности.

К данному типу библиотек относятся:

- `ParMETIS`;
- `KaHIP`;
- `KaMinPar`.

Для запуска ParMETIS используется функция `ParMETIS_V3_PartKway`, реализующая параллельный многоуровневый алгоритм *k-way partitioning*.

В случае KaHIP применяется интерфейс `ParHIPPartitionKWay`, обеспечивающий иерархическое распределённое разбиение графа.

Для библиотеки KaMinPar дополнительно выполняется преобразование внутренних типов данных в специализированные структуры `GlobalNodeID` и `GlobalEdgeID`, используемые библиотекой. Кроме того, реализована поддержка различных конфигурационных контекстов (`default`, `strong`), позволяющих изменять стратегии оптимизации и исследовать влияние параметров библиотеки на качество разбиения.

Библиотека PT-Scotch использует собственное представление распределённого графа в виде структуры `SCOTCH_Dgraph`. В связи с этим в модуле реализован отдельный этап подготовки данных, включающий:

- инициализацию распределённого графа;
- заполнение внутренних структур на основе CSR-представления;
- настройку параметров стратегии разбиения.

После подготовки структуры вызывается функция `SCOTCH_dgraphPart`, выполняющая непосредственно разбиение графа.

Дополнительно реализована поддержка различных стратегий библиотеки:

- `SCOTCH_STRATSPEED`;
- `SCOTCH_STRATQUALITY`;
- `SCOTCH_STRATBALANCE`.

Это позволяет исследовать компромисс между скоростью выполнения алгоритма и качеством получаемого разбиения.

Наиболее сложной частью модуля является интеграция библиотек Zoltan2 и Trilinos, использующих объектно-ориентированную инфраструктуру

распределённых данных.

Для работы алгоритмов Sphynx и PHG реализована процедура преобразования низкоуровневого CSR-представления в объекты Trilinos:

- создание распределённой карты Tpetra::Map;
- построение распределённого графа Tpetra::CrsGraph;
- создание адаптера Zoltan2::XpetraCrsGraphAdapter.

Такой подход позволяет использовать внутренние механизмы распределения данных и параллелизма, реализованные в Trilinos.

Для алгоритма Sphynx дополнительно реализована поддержка настройки спектрального решателя:

- выбор типа лапласиана (combinatorial, generalized);
- выбор предобуславливателя (jacobi, polynomial, muelu);
- настройка параметров LOBPCG.

Для гиперграфового метода PHG реализована возможность переключения между различными стратегиями работы, включая усиленный режим (strong), предусматривающий более глубокое уточнение границ разделов и увеличение числа итераций refinement-этапа.

Для корректного сравнения производительности библиотек в модуле реализована двухуровневая система профилирования с использованием функции MPI_Wtime().

Первая метрика — time_solve — измеряет исключительно время работы внутреннего алгоритма разбиения внутри библиотеки. Перед началом и завершением измерения выполняется синхронизация процессов с помощью MPI_Barrier, что позволяет исключить влияние разброса времени между узлами.

Вторая метрика — time_total — включает полное время обработки:

- 1 подготовку распределённых структур данных;

- 2 преобразование форматов;
- 3 выполнение алгоритма разбиения;
- 4 освобождение памяти.

Такой подход позволяет отдельно оценивать вычислительную сложность алгоритма и накладные расходы, связанные с интеграцией библиотеки в распределённую вычислительную среду.

Архитектура класса `Partition` позволяет динамически изменять параметры алгоритмов без модификации основной логики программы. Это обеспечивает возможность исследования влияния внутренних стратегий библиотек на качество разбиения, балансировку нагрузки и объём межпроцессорных коммуникаций.

Благодаря унифицированной архитектуре модуль обеспечивает корректное и воспроизводимое сравнение алгоритмов различной математической природы в рамках единой экспериментальной среды.

6.3 Модуль сбора и анализа метрик качества

Модуль анализа метрик качества, реализованный в классе `PartitionMetrics`, предназначен для комплексной оценки полученных разбиений графа. В отличие от встроенных средств статистики, предоставляемых отдельными библиотеками разбиения, разработанный модуль выполняет независимый анализ результатов с использованием единых алгоритмов и одинаковых критериев оценки для всех рассматриваемых методов. Это позволяет обеспечить корректность и сопоставимость результатов сравнительного исследования.

Особенностью реализации является необходимость выполнения большинства вычислений в условиях распределённой памяти, когда каждый MPI-процесс обладает информацией только о локальной части графа. В связи с этим значительная часть модуля посвящена организации обмена метаданными между процессами и построению глобальных характеристик разбиения.

Для корректного вычисления коммуникационных и топологических метрик реализован механизм работы с так называемыми «призрачными» вершинами (ghost vertices) [29]. Данный механизм позволяет каждому процессу получать информацию о принадлежности удалённых вершин к разделам графа без необходимости хранения полного графа в памяти.

На первом этапе каждый процесс анализирует локальные рёбра и определяет вершины, принадлежащие другим MPI-процессам. Далее выполняется коллективный обмен данными с использованием операций MPI_Alltoall и MPI_Alltoallv, в ходе которого процессы обмениваются информацией о номерах разделов, назначенных удалённым вершинам.

Полученные данные сохраняются во внутренних структурах:

- ghost_global_ids;
- ghost_part_ids.

Наличие локального кэша удалённых вершин позволяет в дальнейшем выполнять вычисление метрик без дополнительных сетевых обращений, что существенно снижает накладные расходы при анализе больших графов.

Модуль выполняет вычисление дисбаланса как по вершинам, так и по рёбрам графа.

Для каждого раздела вычисляется фактическое число вершин и рёбер, после чего с помощью операции MPI_Allreduce формируются глобальные счётчики по всем процессам. На основе этих данных рассчитывается отклонение от идеального среднего значения.

Для дисбаланса вершин используется отношение $\max(V_i) / (|V|/k)$

где:

- V_i — число вершин в разделе;
- $|V|$ — общее число вершин графа;
- k — количество разделов.

Аналогичным образом вычисляется дисбаланс по рёбрам.

Для оценки объёма межпроцессорных коммуникаций модуль выполняет анализ граничных вершин и структуры взаимодействия между разделами.

Граничной считается вершина, имеющая хотя бы одно ребро, ведущее в другой раздел графа. Подсчёт числа таких вершин позволяет оценить объём данных, требующих синхронизации между процессами.

Кроме того, строится граф соседства разделов, описывающий коммуникационную структуру между доменами. Для хранения междоменных связей используется специализированный тип данных Edge, представляющий пару взаимодействующих разделов.

После локального анализа все найденные связи собираются с помощью коллективных MPI-операций, что позволяет построить глобальную топологию взаимодействия между разделами и вычислить:

- число соседей для каждого раздела;
- среднее число междоменных связей;
- максимальную степень коммуникационного взаимодействия.

Данные характеристики позволяют оценить сложность коммуникационной структуры, возникающей при выполнении распределённых вычислений.

Одной из наиболее сложных реализованных метрик является вычисление количества связных компонент внутри каждого раздела графа. Данная характеристика позволяет определить степень фрагментации доменов, формируемых алгоритмом разбиения.

На первом этапе на каждом процессе выполняется локальный поиск компонент связности с использованием алгоритма обхода графа в ширину (BFS). Анализ проводится только внутри вершин, принадлежащих одному разделу.

Однако локального анализа недостаточно, поскольку одна и та же компонента может быть распределена между несколькими MPI-процессами. Для объединения локальных результатов реализован глобальный этап «склейки»

компонент.

Информация о граничных компонентах передаётся на корневой процесс, где выполняется объединение компонент с помощью структуры данных DSU (Disjoint Set Union) [30].

Использование DSU позволяет эффективно определить глобальные связные компоненты разделов и вычислить итоговое количество независимых фрагментов внутри каждого домена.

Данная метрика является особенно важной для оценки качества разбиения, поскольку сильная фрагментация доменов негативно влияет на локальность данных и может ухудшать сходимость численных методов.

Для последующего анализа результатов реализована система сохранения статистики и топологии разбиения.

Модуль поддерживает несколько форматов вывода:

- текстовые отчёты со статистическими характеристиками;
- CSV-файлы отображения «вершина — раздел»;
- экспорт топологии графа для последующей визуализации.

Текстовые отчёты содержат агрегированные статистические показатели по всем метрикам качества, включая средние значения, размах распределения и стандартное отклонение.

Файлы отображения разделов позволяют использовать результаты декомпозиции во внешних прикладных задачах, а экспорт топологии обеспечивает возможность визуального анализа структуры разбиения в специализированных инструментах, таких как Gephi [31] и Cytoscape [32].

Все операции записи выполняются параллельно с использованием MPI-IO, что обеспечивает масштабируемое сохранение результатов даже для графов с миллионами вершин и рёбер.

7 Результаты тестирования

Для проведения сравнительного анализа программа была выполнена на графе, представленном в виде разреженной матрицы размерности $44\,609 \times 44\,609$, содержащей 2\,014\,701 ненулевых элементов.

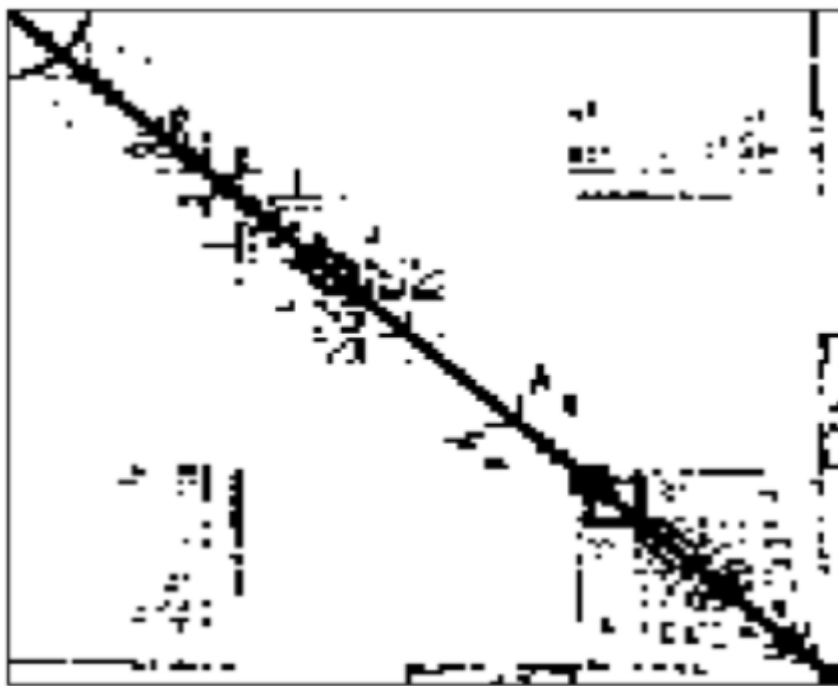


Рис. 1 – Структура исследуемой матрицы.

Тестирование проводилось для библиотек ParMETIS, PT-Scotch, ParHIP, dKaMinPar, Zoltan (PHG) и Zoltan2 (Sphynx). Все алгоритмы запускались в одинаковых условиях с фиксированными параметрами дисбаланса и одинаковым числом разделов, что обеспечивало корректность сравнительного анализа.

7.1 Сравнение алгоритмов по коммуникационным и топологическим метрикам

Одной из основных характеристик качества разбиения является количество разрезанных рёбер (Edge Cut), поскольку именно этот параметр напрямую определяет объём межпроцессорных коммуникаций при распределённых вычислениях.

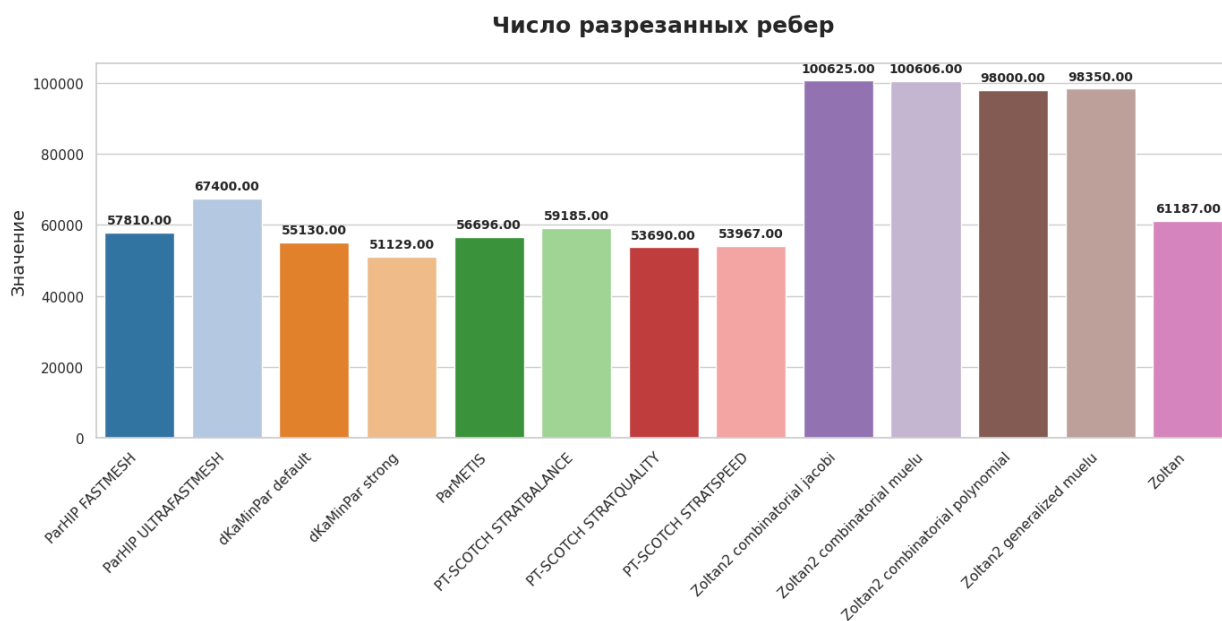


Рис. 2 – Полученные значения метрики числа разрезанных ребер

Наилучший результат по минимизации количества разрезанных рёбер показала библиотека dKaMinPar в конфигурации strong, получив значение 51129. Близкие результаты продемонстрировали:

- PT-Scotch (STRATQUALITY) — 53690;
- dKaMinPar (default) — 55130;
- ParMETIS — 56696.

Библиотеки семейства ParHIP также обеспечили хорошие результаты, однако режим ULTRAFSTMESH ожидаемо ухудшил качество разбиения ради повышения скорости работы.

Наиболее высокий edge cut наблюдался у спектрального метода Zoltan2 Sphynx, где количество разрезанных рёбер составило около 98000–100000. Это

существенно превышает показатели классических многоуровневых алгоритмов. Подобное поведение объясняется тем, что спектральные методы ориентированы прежде всего на глобальную структуру графа, а не на локальную минимизацию числа разрезанных связей.

Аналогичная тенденция наблюдается и для среднего числа граничных вершин (Boundary Vertices).

Лучшие показатели продемонстрировали:

- dKaMinPar strong — 774.7;
- PT-Scotch STRATQUALITY — 782.3;
- Zoltan PHG — 791.2.

Методы Sphynx показали существенно большее число граничных вершин (1330–1410).

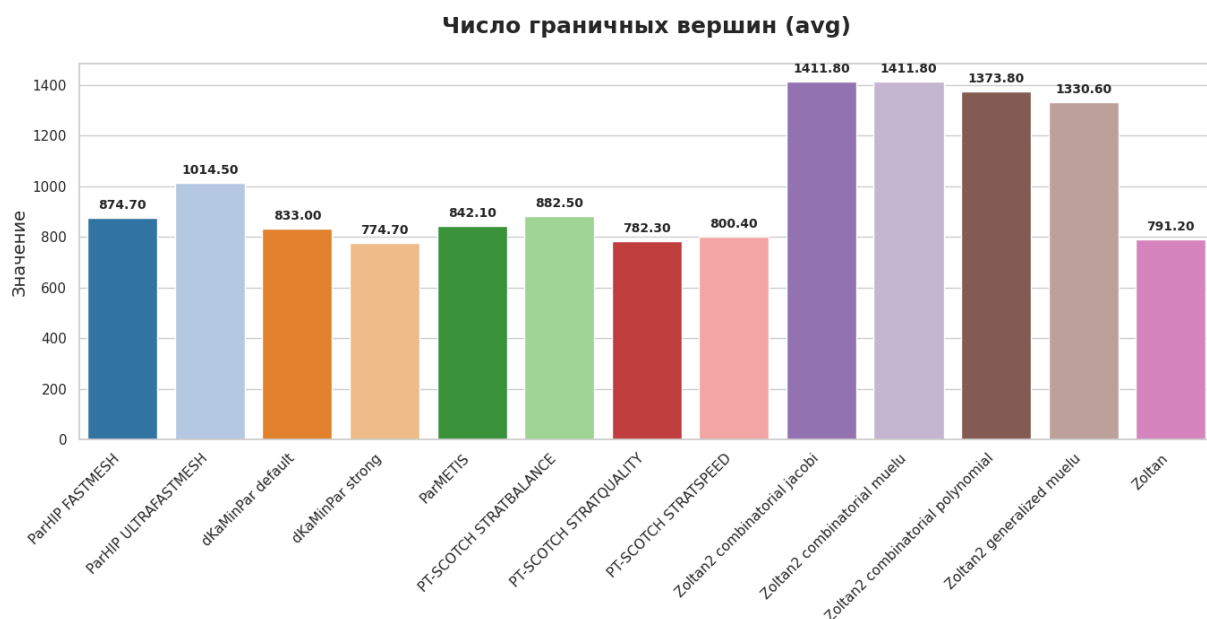


Рис. 3 – Полученные значения метрики числа граничных вершин

При анализе среднего числа соседних разделов (Neighbors Count) наиболее низкие значения также были получены для алгоритмов Sphynx (3.8–4.0). Несмотря на высокий edge cut, спектральный метод формирует более компактную структуру взаимодействия между доменами: каждый раздел взаимодействует с меньшим числом соседних процессов. Это может быть

полезно для некоторых типов MPI-приложений, чувствительных именно к числу коммуникационных партнёров, а не к объёму передаваемых данных.

Среди классических методов хорошие результаты показали:

- ParMETIS — 4.4;
- PT-Scotch STRATSPEED — 4.2;
- dKaMinPar — 5



Рис. 4 – Полученные значения метрики числа соседних кластеров

Анализ связности разделов показал, что большинство многоуровневых алгоритмов формируют практически полностью связные домены. Для ParMETIS, PT-Scotch и Zoltan PHG среднее число компонент связности на раздел близко к единице (1.1–1.2), что свидетельствует о высокой топологической целостности разбиения.

В то же время спектральный метод Sphynx демонстрирует заметную фрагментацию доменов до 2.5 компонент в среднем. Это связано с глобальным характером спектрального анализа, при котором вершины могут группироваться по спектральному сходству даже при отсутствии сильной локальной связности.



Рис. 5 – Полученные значения метрики числа компонент связности

7.2 Сравнение алгоритмов по балансировке нагрузки

Балансировка нагрузки является одной из ключевых характеристик качества разбиения, поскольку сильный дисбаланс приводит к неравномерной загрузке MPI-процессов и возникновению простоев при синхронизации вычислений.

Наилучшие результаты по балансировке вершин продемонстрировали алгоритмы библиотеки Zoltan2 Sphynx и стратегия STRATBALANCE библиотеки PT-Scotch. Для них различие между максимальным и минимальным числом вершин в разделах составило всего 1 вершину, а стандартное отклонение практически равно нулю.

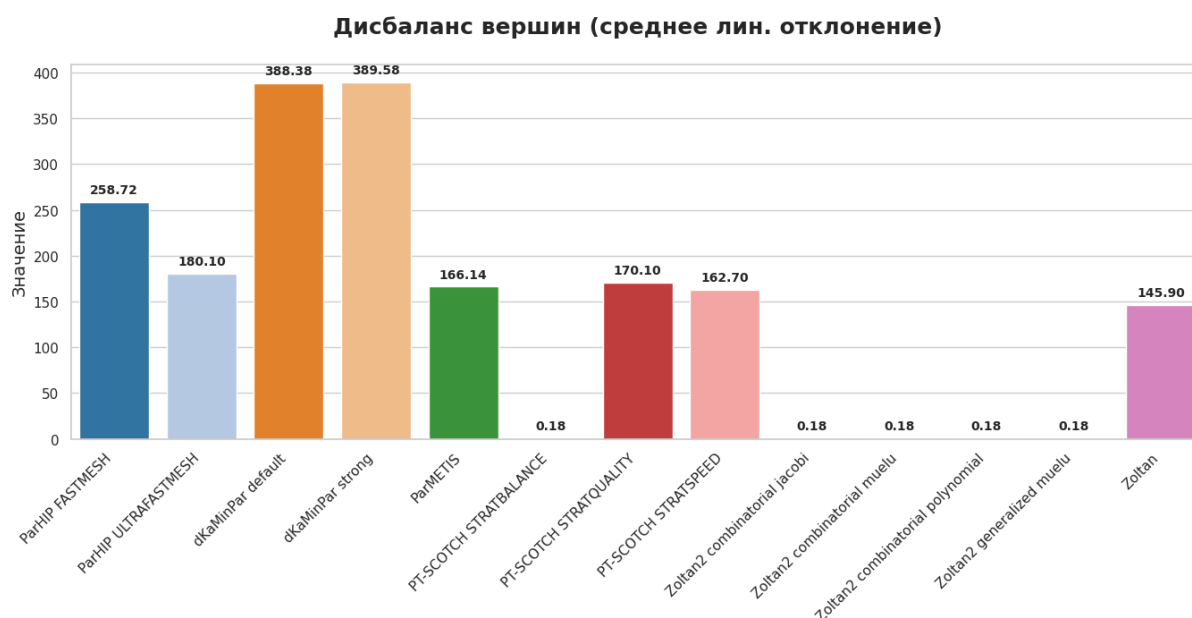


Рис. 5 – Полученные значения метрики дисбаланса вершин

Подобное поведение объясняется тем, что спектральный метод и балансировочные стратегии PT-Scotch явно ориентированы на строгое соблюдение ограничений по размеру разделов.

Классические многоуровневые методы также обеспечили хорошую балансировку:

- ParMETIS = 166;
- ParHIP ULTRAFASTMESH = 180;

- PT-Scotch STRATQUALITY = 170.

Наибольший дисбаланс по вершинам наблюдался у dKaMinPar, где разница между разделами превышала 2200 вершин. Однако при этом библиотека обеспечивала лучшее качество по edge cut, что демонстрирует классический компромисс между балансировкой и минимизацией коммуникаций.

При анализе распределения рёбер ситуация несколько изменилась. Наиболее равномерное распределение рёбер показали методы Sphynx:

- generalized muelu
- combinatorial polynomial

Близкие результаты продемонстрировали:

- PT-Scotch STRATBALANCE
- ParMETIS .

dKaMinPar и ParHIP показали существенно больший дисбаланс по рёбрам, что объясняется их ориентацией преимущественно на минимизацию edge cut и локализацию связей внутри доменов.

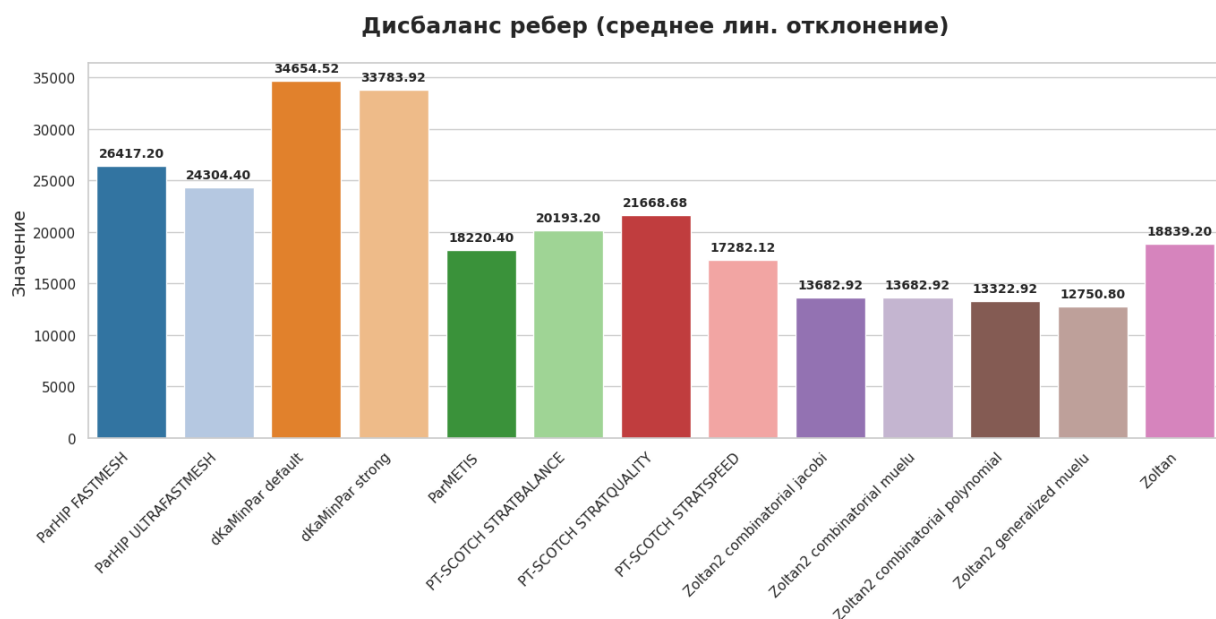


Рис. 6 – Полученные значения метрики дисбаланса ребер

Таким образом, результаты показывают, что различные библиотеки используют разные приоритеты оптимизации:

- Sphynx и PT-Scotch ориентированы на строгую балансировку нагрузки;
- dKaMinPar и ParHIP делают акцент на снижении коммуникационных затрат;
- ParMETIS демонстрирует компромисс между качеством разбиения и балансировкой.

7.3 Сравнение временных характеристик

Анализ временных характеристик показал существенное различие между библиотеками как по скорости выполнения непосредственно алгоритма разбиения, так и по совокупному времени работы.

Наиболее быстрым решением оказался ParMETIS, продемонстрировавший время разбиения около 0.033 секунды. Это подтверждает статус ParMETIS как одной из наиболее оптимизированных библиотек для многоуровневого partitioning.

Высокую производительность также показали:

- ParHIP ULTRAFASTMESH — 0.082 с;
- ParHIP FASTMESH — 0.119 с.

KaMinPar оказался заметно медленнее:

- default — 0.326 с;
- strong — 0.581 с.

Увеличение времени в режиме strong связано с использованием более агрессивных стратегий уточнения границ и дополнительными итерациями refinement-этапа.

Библиотека PT-Scotch продемонстрировала наиболее высокое время работы среди классических алгоритмов:

- от 1.1 до 1.8 секунды в зависимости от стратегии.

Это объясняется сложной системой внутренних эвристик и более глубокими этапами оптимизации качества разбиения.

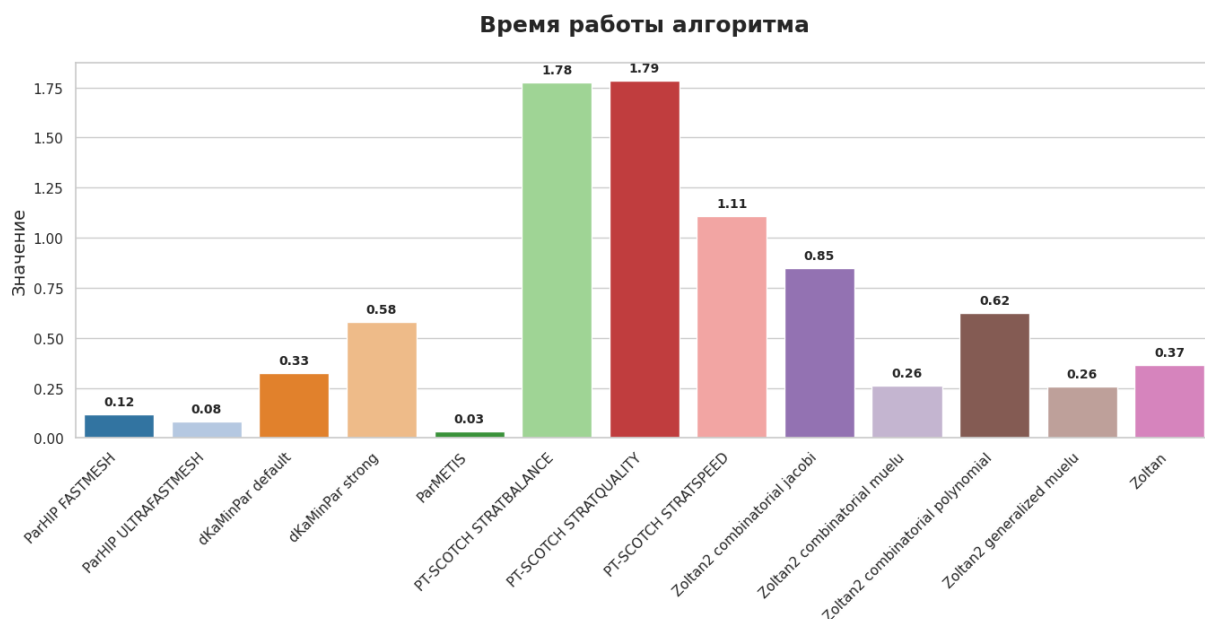


Рис. 7 – Полученные значения метрики времени работы

Следует отметить, что для рассматриваемого графа время подготовки данных оказалось крайне малым. Разница между `time_total` и `time_solve` практически отсутствует для большинства библиотек, что связано с относительно небольшим размером графа и эффективной реализацией конвертации CSR-структур.

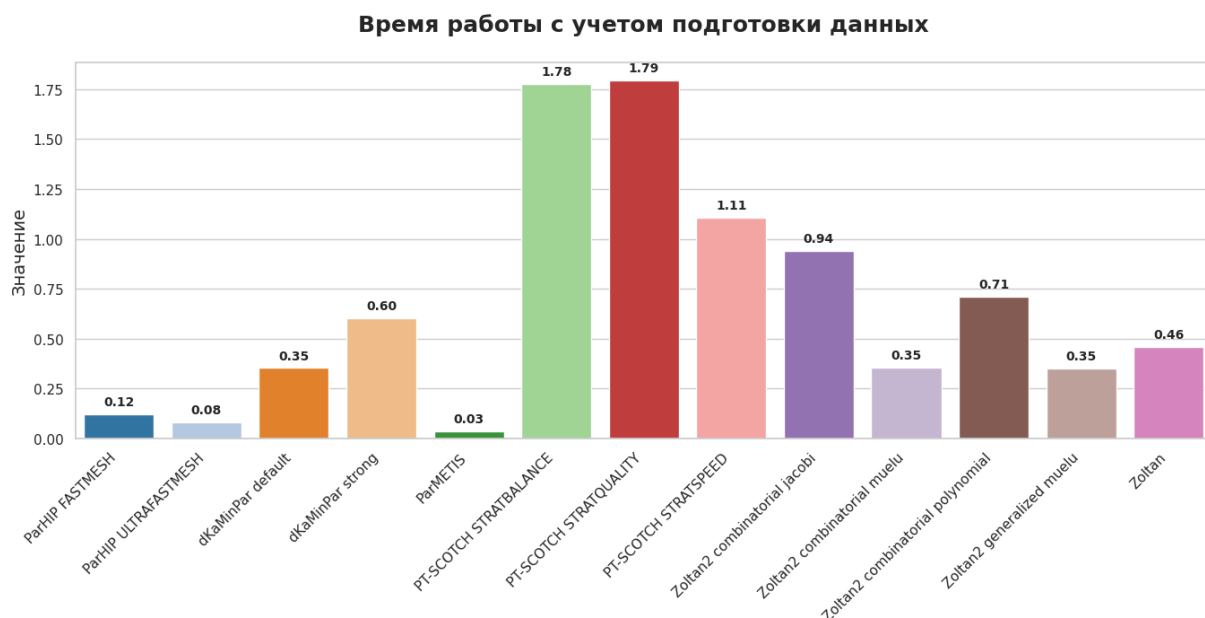


Рис. 8 – Полученные значения метрики времени работы с учетом подготовки данных

7.4 Итоги сравнения

Проведённое исследование показало, что различные библиотеки разбиения графов ориентированы на разные критерии оптимизации, вследствие чего качество их работы существенно зависит от рассматриваемой группы метрик.

С точки зрения коммуникационных характеристик наиболее эффективные результаты продемонстрировали многоуровневые алгоритмы. Лучшие значения по количеству разрезанных рёбер, числу граничных вершин и общей компактности разделов показали dKaMinPar, ParMETIS, PT-Scotch и ParHIP. Особенно выделился dKaMinPar в конфигурации strong, обеспечивший минимальный edge cut среди всех протестированных библиотек.

ParMETIS продемонстрировал наиболее сбалансированное поведение среди алгоритмов декомпозиции графов: библиотека одновременно обеспечила низкий объём коммуникаций, хорошую связность разделов и очень высокую скорость работы. ParHIP, в свою очередь, показал хороший компромисс между качеством разбиения и производительностью, особенно в режиме ULTRAFASTMESH, ориентированном на быстрое получение результата.

PT-Scotch продемонстрировал высокое качество разбиения и хорошие показатели балансировки, однако достигалось это ценой существенно большего времени работы. При использовании стратегии STRATQUALITY библиотека обеспечивала результаты, близкие к KaMinPar, особенно по edge cut и числу граничных вершин.

Гиперграфовый метод Zoltan PHG показал стабильные результаты по большинству коммуникационных и топологических метрик. Несмотря на то, что он уступил лучшим многоуровневым алгоритмам по минимизации edge cut, метод обеспечил хорошую связность разделов и умеренный уровень коммуникаций. Это подтверждает эффективность гиперграфового подхода для задач, где важно учитывать структуру вычислительных зависимостей.

Спектральный метод Zoltan2 Sphynx продемонстрировал существенно иное поведение по сравнению с классическими методами. С одной стороны, алгоритм обеспечил практически идеальную балансировку вершин и хорошее распределение рёбер между разделами. Кроме того, Sphynx формировал разделы

с меньшим числом соседних доменов, что уменьшает количество коммуникационных партнёров в MPI-приложениях. С другой стороны, спектральный подход показал значительно худшие результаты по edge cut, числу граничных вершин и связности доменов. Полученные разделы оказались более фрагментированными, а объём межпроцессорных коммуникаций — существенно выше, чем у многоуровневых алгоритмов. Это связано с тем, что Sphynx ориентирован прежде всего на глобальную спектральную структуру графа и строгую балансировку нагрузки, а не на локальную минимизацию разрезов.

Исследование временных характеристик подтвердило высокую эффективность классических многоуровневых методов. Наиболее быстрым решением оказался ParMETIS, показавший минимальное время работы среди всех библиотек. Очень высокую производительность также продемонстрировали алгоритмы ParHIP. Наиболее медленным среди классических методов оказался PT-Scotch.

8 Заключение

В данной работе проведено исследование и сравнительный анализ современных библиотек разбиения графов, применяемых в распределённых вычислительных системах на базе MPI, включая ParMETIS, PT-Scotch, ParHIP, dKaMinPar, Zoltan (PHG) и Zoltan2 (Sphynx).

Актуальность выбранного направления обусловлена широким использованием графовых моделей в задачах высокопроизводительных вычислений и необходимостью эффективного распределения данных между вычислительными узлами. Появление новых алгоритмов и библиотек декомпозиции графов делает задачу сравнения особенно важной.

В рамках работы была написана программа на языке C++, предназначенная для тестирования различных библиотек разбиения графов.

В ходе реализации были решены следующие задачи:

- Реализовано параллельное чтение графов в формате CSR с использованием MPI-IO;
- Реализованы средства предварительной обработки графов;
- Разработан единый интерфейс взаимодействия с различными библиотеками разбиения графов;
- Реализован модуль вычисления метрик качества разбиения;
- Выполнен последующий сравнительный анализ полученных данных.

Полученные результаты показали, что современные библиотеки разбиения обладают различными сильными сторонами и ориентированы на разные критерии оптимизации разбиения графа.

9 Список литературы

1. Garey M. R., Johnson D. S. Computers and Intractability: A Guide to the Theory of NP-Completeness. — W. H. Freeman & Co., 1979
2. Библиотека ParMETIS [Электронный ресурс].- Электрон. дан. - URL: <https://github.com/KarypisLab/ParMETIS> (дата обращения 30.01.2026)
3. Библиотека Scotch [Электронный ресурс].- Электрон. дан. - URL: <https://github.com/live-clones/scotch> (дата обращения 30.01.2026)
4. Библиотека KaHIP [Электронный ресурс].- Электрон. дан. - URL: <https://github.com/kahip/kahip> (дата обращения 30.01.2026)
5. Библиотека KaMinPar [Электронный ресурс].- Электрон. дан. - URL: <https://github.com/KaHIP/KaMinPar> (дата обращения 30.01.2026)
6. Библиотека Zoltan в составе пакета Trilinos [Электронный ресурс].- Электрон. дан. - URL: <https://github.com/trilinos/Trilinos> (дата обращения 30.01.2026)
7. Библиотека Zoltan2 в составе пакета Trilinos [Электронный ресурс].- Электрон. дан. - URL: <https://github.com/trilinos/Trilinos> (дата обращения 30.01.2026)
8. Стандарт интерфейса MPI (Message Passing Interface) [Электронный ресурс].- Электрон. дан. - URL: <https://www.mpi-forum.org/docs/> (дата обращения 30.01.2026)
9. Формат хранения матриц CSR (Compressed Sparse Row) [Электронный ресурс].- Электрон. дан. - URL: <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-dpcpp/2024-0/sparse-storage-formats.html> (дата обращения 30.01.2026)
10. Документация языка C++ [Электронный ресурс].- Электрон. дан. - URL: <https://en.cppreference.com/w/> (дата обращения 30.01.2026)
11. Karypis G., Kumar V. A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. SIAM Journal on Scientific Computing. 1998

12. Devine K. D., Boman E. G., Heaphy R. T. et al. Parallel Hypergraph Partitioning for Scientific Computing. Proceedings of the 20th IEEE International Parallel and Distributed Processing Symposium (IPDPS). 2006.
13. Pelissier C., Lumsdaine A. Sphynx: A Parallel Spectral Graph Partitioner. Technical Report. 2017.
14. Библиотека METIS [Электронный ресурс].- Электрон. дан. - URL: <https://github.com/KarypisLab/METIS> (дата обращения 30.01.2026)
15. Zienkiewicz O. C., Taylor R. L., Zhu J. Z. The Finite Element Method: Its Basis and Fundamentals. — Elsevier, 2005.
16. Schloegel K., Karypis G., Kumar V. Graph Partitioning for High Performance Scientific Computing. 2000.
17. Sanders P., Schulz C. Engineering Multilevel Graph Partitioning Algorithms. Proceedings of the 19th European Conference on Algorithms (ESA). 2011.
18. Fiduccia C. M., Mattheyses R. M. A Linear-Time Heuristic for Improving Network Partitions. Proceedings of the 19th Design Automation Conference. 1982.
19. Schlag S., Schulz C., Seemaier D. et al. High-Quality Distributed Graph Partitioning. Proceedings of the 29th Annual European Symposium on Algorithms (ESA). 2021.
20. Документация по архитектуре Non-Uniform Memory Access (NUMA) [Электронный ресурс].- Электрон. дан. - URL: https://docs.kernel.org/admin-guide/mm/numa_memory_policy.html (дата обращения 30.01.2026)
21. Devine K. D., Rajamanickam S., Boman E. G. et al. Multi-Jagged: A Scalable Parallel Partitioning Algorithm. IEEE Transactions on Parallel and Distributed Systems. 2016.
22. Knyazev A. V. Toward the Optimal Preconditioned Eigensolver: Locally Optimal Block Preconditioned Conjugate Gradient Method. SIAM Journal on Scientific Computing. 2001.

23. Документация пакета Anasazi [Электронный ресурс].- Электрон. дан. - URL: <https://github.com/trilinos/Trilinos/tree/master/packages/anasazi> (дата обращения 30.01.2026)
24. Документация пакета MueLu — Algebraic Multigrid [Электронный ресурс].- Электрон. дан. - URL: <https://github.com/trilinos/Trilinos/tree/master/packages/muelu> (дата обращения 30.01.2025)
25. Von Luxburg U. A tutorial on spectral clustering. Statistics and computing. 2007.
26. Bader D. A., Meyerhenke H., Sanders P. et al. Graph Partitioning and Graph Clustering: 10th DIMACS Implementation Challenge Workshop. — American Mathematical Society, 2013.
27. Walshaw C. The Graph Partitioning Archive [Электронный ресурс].- Электрон. дан. - URL: <http://chriswalshaw.co.uk/partition/> (дата обращения 30.01.2026)
28. Документация пакета Tpetra [Электронный ресурс].- Электрон. дан. - URL: <https://github.com/trilinos/Trilinos/tree/master/packages/tpetra/> (дата обращения 30.01.2026)
29. Eijkhout V. Introduction to High Performance Scientific Computing. — University of Texas at Austin, 2016.
30. Tarjan R. E. Efficiency of a Good But Not Linear Set Union Algorithm. Journal of the ACM. 1975.
31. Инструментарий визуализации Gephi [Электронный ресурс].- Электрон. дан. - URL: <https://gephi.org/> (дата обращения 30.01.2026)
32. Платформа для анализа графов Cytoscape [Электронный ресурс].- Электрон. дан. - URL: <https://cytoscape.org/> (дата обращения 30.01.2026)